How Airbnb learned to personalise search with embeddings [Part 2]

A practical walk-through of how short-term behaviour and long-term preferences were embedded, combined, and used in a production search ranking system.

JOSE PARREÑO GARCIA
MAR 12, 2026

8 1 2 Share

This week, I want to pick up the second part of Airbnb's personalisation paper.

In Part 1, we covered how Airbnb built high quality embeddings for listings and accommodations, but only using clicks sequences. It was super interesting to see how only using these user interactions (and not adding any extra listing information such as geography, prices or accommodation type) managed to create really solid clusters.

In this second part, we will cover how Airbnb built embeddings that described the actual travellers. And, how these embedding were used real-time in production grade systems, adapting to what the user had recently clicked on the page vs what they had booked a long time ago.

We will cover some hard design problems such as:

- How to model guest intent when user histories are sparse and unreliable.
- How to turn rare bookings into something learnable.
- And how embeddings stop being a standalone artefact and instead become *real-time ranking features* inside a production search system.

If you care about real-time personalisation at scale — where models must react to what users just did, while still respecting what they did months ago — this is where the paper becomes genuinely interesting.

Where can you find the paper?

The paper is titled "**Real-time Personalization using Embeddings for Search Ranking at Airbnb**", authored by *Mihajlo Grbovic* and *Haibin Cheng*. It was presented at KDD 2018 and is publicly available online.

Read it here → Real-time Personalization using Embeddings for Search Ranking at Airbnb

Paper breakdown in 2 parts

- ✅ How Airbnb learned to understand listings via embeddings [Part 1]
- 📌 How Airbnb learned to personalise search with embeddings [Part 2] ***(this post!)***

What will this blog post cover?

This post focuses on the second half of Airbnb's personalisation system: how guest intent is represented, learned, and used inside a real-time ranking pipeline.

More specifically, we will cover:

- **Why user modelling is fundamentally harder than listing modelling in travel.** Travellers book infrequently, often anonymously, and rarely build long, reliable histories. We will look at why classic user embeddings break down under these constraints, and why Airbnb could not rely on user IDs alone.
- **How Airbnb avoided user-level sparsity with a bucketing strategy.** Instead of learning one embedding per user, Airbnb introduced *user types* and *listing types* — many-to-one mappings that trade individual precision for learnability, stability, and memory efficiency.
- **How booking histories were transformed into learnable sequences.** We will see how sparse booking events become training data by alternating user

types and listing types, allowing skip-gram style objectives to work even when bookings are rare.

- **Why host rejections matter — and how they were injected into the embedding space.**
  Unlike clicks, bookings contain explicit negative feedback. Airbnb used host rejections as negative signals during training, shaping embeddings to reduce future mismatches between guests and hosts.
- **From standalone embeddings to behavioural signals.**
  Clicks, skips, wishlists, and bookings were treated very differently, yet brought together through embedding similarity features that adapt in real time during a search session.
- **How embeddings moved from representations to ranking features.**
  Rather than replacing the ranking model, embeddings became inputs to it — producing interpretable, high-impact features that improved bookings without increasing rejections.

Why user modelling is fundamentally harder than listing modelling in travel.

If learning embeddings for listings felt relatively easy to tackle, users quickly reveal why personalisation in travel is much harder.

User behaviour in travel is sparse by nature. Most travellers book once or twice a year, many appear only once, and a significant share never log in at all. Even when booking history exists, it may be so old that the context has completely changed. A past booking might reflect a family holiday, while the next search could be for a short romantic weekend — preferences that look nothing alike from a modelling perspective.

At the same time, Airbnb already had something that worked well (see part 1 of this paper deep dive). Listing embeddings trained on click sessions were excellent at capturing *short-term intent* — what the user seems to be looking for right now, within a single search session. What they lacked was a way to complement that signal with something longer-term, without falling into the usual traps of sparsity and overfitting. For example, *"given a user who is currently searching for a listing in Los Angeles, and has made past bookings in New York and London, it would be useful to recommend listings that are similar to those previously booked ones"*.

The rest of this post is about how they navigated that trade-off.

How Airbnb avoided user-level sparsity with a bucketing strategy.

Given the constraints we just described, a natural idea would be to learn an embedding for each individual user and update it over time as bookings accumulate.

Airbnb explicitly chose ***not to do that.***

1. Learning user-level embeddings sounds appealing in theory, but in practice it runs into several problems at once. Most users simply do not generate enough bookings to learn a meaningful representation. Many users would end up with embeddings trained on a single event, which is indistinguishable from noise.
2. On top of that, storing and updating embeddings for millions of users introduces a non-trivial memory and infrastructure cost, especially in systems that need to compute ranking features in real time.

Instead, Airbnb took a step back and asked a more pragmatic question.

> What if, rather than representing *users*, we represented *types of users*?

Building user type buckets

Rather than learning one embedding per user ID, Airbnb groups users into **user types** based on a set of observable attributes. These set of attributes are well specified in table 4.

Table 4: Mappings of user meta data to user type buckets

Features that don't require booking history

Buckets	1	2	3	4	5	6	7	8
Market	SF	NYC	LA	HK	PHL	AUS	LV	...
Language	en	es	fr	jp	ru	ko	de	...
Device Type	Mac	Msft	Andr	Ipad	Tablet	Iphone	...	
Full Profile	Yes	No						
Profile Photo	Yes	No						
Num Bookings	0	1	2-7	8+				
$ per Night	<40	40-55	56-69	70-83	84-100	101-129	130-189	190+
$ per Guest	<21	21-27	28-34	35-42	43-52	53-75	76+	
Capacity	<2	2-2.6	2.7-3	3.1-4	4.1-6	6.1+		
Num Reviews	<1	1-3.5	3.6-10	> 10				
Listing 5 Star %	0-40	41-60	61-90	90+				
Guest 5 Star %	0-40	41-60	61-90	90+				

Booking history features for longer term known users

Many individual users map to the same user type, turning a sparse, long-tailed problem into something far more learnable. Whilst one might argue that the number of possible user types is still large (easily in the hundreds of thousands once you account for markets, languages, devices, and behavioural buckets), it is still a much more manageable problem compared to modelling *individual* users. The space is bounded, shared across many travellers, and grows slowly compared to the user base itself. Crucially, new users do not introduce new user types; they are simply mapped

into an existing one. This makes the approach far more tractable than maintaining embeddings for millions of sparse user IDs, especially under real-time and cold-start constraints.

Building listing type buckets

With this in place, Airbnb applied the same idea on the supply side.

In Part 1, we saw how listing embeddings worked well for short-term signals such as clicks. Here, however, the focus shifts to long-term signals derived from bookings. These are also sparse: many listings accumulate very few bookings, and new listings continuously enter the platform. Mapping listings into **listing types** therefore becomes a natural extension of the same strategy.

Table 3: Mappings of listing meta data to listing type buckets

Buckets	1	2	3	4	5	6	7	8
Country	US	CA	GB	FR	MX	AU	ES	...
Listing Type	Ent	Priv	Share					
$ per Night	<40	40-55	56-69	70-83	84-100	101-129	130-189	190+
$ per Guest	<21	21-27	28-34	35-42	43-52	53-75	76+	
Num Reviews	0	1	2-5	6-10	11-35	35+		
Listing 5 Star %	0-40	41-60	61-90	90+				
Capacity	1	2	3	4	5	6+		
Num Beds	1	2	3	4+				
Num Bedrooms	0	1	2	3	4+			
Num Bathroom	0	1	2	3+				
New Guest Acc %	<60	61-90	>91					

With table 3 and table 4, any user and any listing can be mapped as a categorical bucket. Below you can read the examples provided by Airbnb in their paper:

- Entire Home listing from US that has a 2 person capacity, 1 bed, 1 bedroom & 1 bathroom, with Average Price Per Night of $60.8, Average Price Per Night Per Guest of $29.3, 5 reviews, all 5 stars, and 100% New Guest Accept Rate would map into listing_type = **US_lt1_pn3_pg3_r3_5s4_c2_b1_bd2_bt2_nu3**.
- User from San Francisco with MacBook laptop, English language settings, full profile with user photo, 83.4% average, Guest 5 star rating from hosts, who has

made 3 bookings in the past, where the average statistics of booked listings were $52.52 Price Per Night, $31.85 Price Per Night Per Guest, 2.33 Capacity, 8.24 Reviews and 76.1% Listing 5 star rating, the resulting user_type = **SF_lg1_dt1_fp1_pp1_nb1_ppn2_ppд3_c2_nr3_l5s3_g5s3**

How did this many-to-one mapping help?

1. First, it dramatically increases the amount of data available per embedding, making learning feasible even when bookings are rare.
2. Second, it keeps the total number of embeddings bounded and manageable, which matters when those embeddings must be loaded into memory and queried during live search.

Of course, this comes with an explicit trade-off. Bucketing smooths over individual differences. Two users with different motivations may end up sharing the same user type, and two listings with subtle differences may collapse into the same listing type. Airbnb accepted this loss of precision in exchange for stability, learnability, and scalability.

This choice sets the foundation for everything that follows. Once users and listings are represented at the level of types rather than IDs, bookings can be turned into sequences in a very similar way to how clicks were turned into sequences in the first blog post.

In the next section, we will look at how Airbnb transformed sparse booking histories into training data using this representation.

How booking histories were transformed into learnable sequences.

Click sequences were easy

In Part 1, constructing training sequences from click data was conceptually straightforward.

A search session could be defined using simple rules: a user issues a query, clicks on a few listings, and after a period of inactivity the session ends. Once the session ends, then the listings which were clicked are ordered and this becomes a “sequence”.

```
Listing 1 → Listing 7 → Listing 3 → Listing 9 → Listing 3
```

Each of the “Listing X” becomes a token. And a sliding window moves across the sequence, and each listing takes a turn as the centre token. Nearby listings define its context. For example; if *Listing 7* is the centre, then *Listing 1* and *Listing 3* are the context pairs.

Because of the skip-gram sequencing with clicks is pretty easy and has abundant data, the hard part in Part 1 was not defining the sequences, but refining the signal. Airbnb spent most of their effort adjusting the objective — adding bookings as global context, correcting negative sampling for congregated search, and handling cold-start listings.

With booking data, that balance shifts.

The difficulty is no longer the optimisation or the loss function, but how to construct sequences that are informative enough in the first place.

Why booking event needs a different sequence structure

A booking is *the* strongest signal of relevance.

But, we also know that these are sparse. Airbnb could not simply create sequences of booked listings ordered one after another, because very few users had multiple bookings. And even when they did, those bookings were often spread over long periods of time, making the definition of a meaningful “session” extremely difficult.

- The first part of the solution, as we saw earlier, was to categorise both users and listings into buckets.
- The second part was to rethink the representation of the sequence itself.

Instead of representing a booking as a single token, Airbnb represented it as a **pair of tokens**:

```
(user_type, listing_type)
```

In Part 1, a click corresponded to one token in the sequence. In Part 2, a booking corresponds to **two tokens**, one for the user side and one for the listing side.

Let us make this concrete with a simple example. Imagine a traveller who books three trips over a couple of years. In the real world, that history might look like this:

Time	City	User	Listing
2022	NYC	Jose	Loft A
2023	London	Jose	Flat B
2024	LA	Jose	House C

Airbnb would take this raw data and bucket it to *user* and *listing types*. For example:

- `U1` = “urban, English, desktop, mid-price, frequent traveller”
- `L5` = “entire home, medium price, high reviews”

- L8 = “central flat, smaller capacity”
- L2 = “larger house, family-friendly”

Each booking is first represented as a pair:

```
Booking pairs

(U1, L5), (U1, L8), (U1, L2)
```

These pairs are then **flattened** into a single alternating sequence:

```
Flattening

U1 → L5 → U1 → L8 → U1 → L2
```

From the model’s point of view, this is just a sequence of tokens. There is no special notion of “pair” at training time. What matters is that user types and listing types now live in the **same sequence**, and therefore in the **same embedding space**.

Now, imagine that we are moving the skip-gram window of size 1. Two situations can occur:

- **For case A:** This produces training signals that encourage the model to place U1 close to L5 and L8 in the embedding space. Intuitively, this captures the idea: "*Users like this tend to book listings like those.*"

```
      _____________________
     |                     |
U1 → | L5   →   U1   →  L8 | → U1 → L2
     |_____________________|
        |       |        |
     context  centre  context
```

- **For case B:** This pulls L8 closer to U1, capturing the complementary relationship: "*Listings like this tend to be booked by users like those.*"

```
          ___________________
         |                   |
U1 → L5 → |  U1   →  L8   →  U1 | → L2
         |___________________|
            |       |        |
         context  centre  context
```

How the equations fit together

At this point, the paper introduces new equations, which can make this section look more complicated than it really is. If you are comfortable with **Equation** (**3**) from Part 1, you already understand the core of what is happening here. The only difference is *which token happens to sit at the centre of the skip-gram window.*

In Part 1, Airbnb trained listing embeddings using the standard skip-gram objective with negative sampling:

$$\max_\theta \sum_{(l,c)\in D^+} \log \sigma\left(v_c^\top v_l\right) \;+\; \sum_{(l,c)\in D^-} \log \sigma\left(-v_c^\top v_l\right)$$

Where:

- `l` is the **centre listing**
- `c` is a **context listing**
- `vl`, `vc` are their embedding vectors
- `D+` contains positive (co-occurring) pairs
- `D−` contains negative (randomly sampled) pairs
- `σ(·)` is the sigmoid function

If this looks unfamiliar, it is worth revisiting Part 1, as nothing fundamentally new is introduced beyond this point.

In Part 2, Airbnb trains **one single skip-gram model** over a **mixed vocabulary** that contains:

- **User types → equation** (6)
  Here, `ut` is a **user type**; `c` can be either a **listing type** or another **user type** from the context window

$$\max_\theta \sum_{(u_t,c)\in D^+} \log \sigma\left(v_c^\top v_{u_t}\right) \; + \; \sum_{(u_t,c)\in D^-} \log \sigma\left(-v_c^\top v_{u_t}\right)$$

- **Listing types → equation** (7)
  Here, `lt` is a **listing type**, `c` is drawn from nearby **user types** or **listing types**

$$\max_\theta \sum_{(l_t,c)\in D^+} \log \sigma\left(v_c^\top v_{l_t}\right) \; + \; \sum_{(l_t,c)\in D^-} \log \sigma\left(-v_c^\top v_{l_t}\right)$$

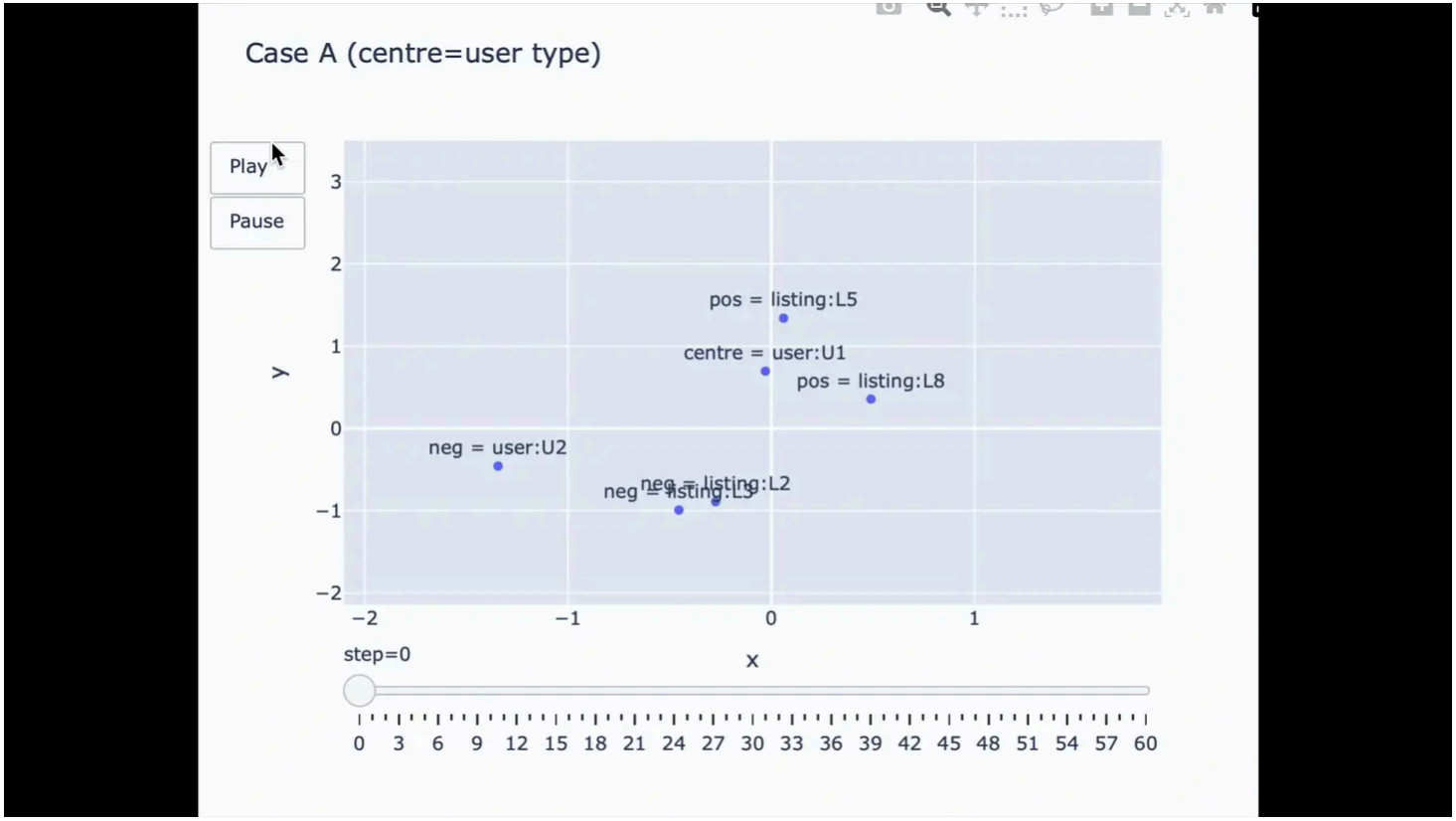
For clarity, the negative sample pairs `D−` for are randomly sampled across the whole user type space for `ut` and across the whole listing type space for `lt`.

Seeing this with a simulation.

The following simulations show the same skip-gram objective we explored in Part 1, now operating over a mixed sequence of **user types** and **listing types**.

In **Case A**, the centre of the window is a user type. As optimisation progresses, the user embedding is pulled closer to the listing types that appear in its booking context,

while randomly sampled negatives are pushed away. Geometrically, the user embedding begins to represent a *taste profile*: it moves towards the kinds of listings this user type tends to book.



In **Case B**, the centre of the window is a listing type. The same pull-and-push dynamics apply, but the interpretation changes. The listing embedding is now pulled

towards the user types that tend to book it. In this case, the embedding starts to represent a *target audience* rather than a preference.

What matters is that the optimisation itself does not change. The same objective, the same gradients, and the same embedding space are reused. Only the identity of the centre token changes, and with it, the meaning of the update. This shared geometry is

what allows Airbnb to later compute meaningful similarities between users and listings, and to inject those similarities directly into the ranking model in real time.

Why host rejections matter — and how they were injected into the embedding space.

Bookings tell us which user types and listing types are compatible, and skip-gram turns those co-occurrences into geometric proximity.

But, bookings also contain something we have not used yet. On Airbnb, a booking does not happen automatically. **Hosts can accept or reject a request**. When a rejection occurs, it is an explicit indication that a particular user–listing combination did *not* work. Ignoring this signal would mean learning only from what succeeded, while repeatedly rediscovering the same failures.

Airbnb repeated the strategy in equation (5) adapting it slightly to the explicit negative examples. The idea is to extend the existing skip-gram formulation with an additional term that treats rejections as **explicit negatives**. Conceptually, this term applies a force in the opposite direction of bookings: when a user type and a listing type are involved in a rejection, their embeddings are pushed apart.

Mathematically, this shows up as Equations (8) and (9) in the paper. They are direct extensions of Equations (6) and (7), with one additional component corresponding to rejected booking attempts. The optimisation remains unchanged; the model simply learns from one more kind of context.

Equation (8)

$$\max_\theta \sum_{(u_t,c)\in D^+} \log \sigma\left(v_c^\top v_{u_t}\right) \;+\; \sum_{(u_t,c)\in D^-} \log \sigma\left(-v_c^\top v_{u_t}\right) \;+\; \sum_{(u_t,r)\in D^r} \log \sigma\left(-v_r^\top v_{u_t}\right)$$

where:

- `r` is a ***listing type*** **involved in a rejected booking**
- `Dr` contains explicit rejection pairs
- The additional term introduces a *repulsive force* between incompatible user–listing pairs

Equation (9)

$$\max_\theta \sum_{(l_t,c)\in D^+} \log \sigma\left(v_c^\top v_{l_t}\right) \;+\; \sum_{(l_t,c)\in D^-} \log \sigma\left(-v_c^\top v_{l_t}\right) \;+\; \sum_{(l_t,r)\in D^r} \log \sigma\left(-v_r^\top v_{l_t}\right)$$

where:

- `r` is a ***user type*** **involved in a rejected booking**

If you want more details on how this extra negative pull works, I break it down step by step in post 1.

This matters because it injects host-side preferences directly into the representation. The embedding space no longer encodes only guest intent, but a negotiated compatibility between guests and hosts — exactly what a two-sided marketplace needs.

At this point, we have covered in the first post short term optimisations; and, in this second post, long term optimisations. But how are they combined together?

From standalone embeddings to behavioural signals

At this point, it is worth pausing to remind that embeddings at Airbnb were already useful *on their own*.

In Part 1, we saw how listing embeddings trained on click sequences could be used directly via cosine similarity to power *Similar Listings*. Given a listing, Airbnb simply retrieved its nearest neighbours in the embedding space and surfaced them to the user.

That direct use is powerful, but it is also limited.

- Similarity between two listings answers a very specific question: *"What looks like this?"*
- Personalisation, however, asks a broader one: *"What does this user want right now, and what do they usually prefer?"*

To answer that, Airbnb does not rely on a single embedding. Instead, they treat embeddings as raw material from which richer behavioural signals can be constructed. In other words, Airbnb used these skip-gram models to also use them as "feature engineering".

Aggregating user behaviour: from individual listings to intent

A user does not interact with just one listing. They click several listings, skip others, wishlist some, and occasionally book. Each of those listings already has an embedding (calculated with the skip-gram model from equations 8 and 9). What Airbnb does is compute a **centroid of these embeddings**.

To make this concrete, imagine a very simple embedding space with just two dimensions, so we can draw it on a plane. Suppose that, during a search session in Paris, a user clicks on three listings:

```
# Each listing represents a clicked listing.
L1 = (1.0, 2.0)
L2 = (2.0, 1.5)
L3 = (1.5, 2.5)

# Airbnb aggregates this short-term behaviour by computing the centroid:
Click centroid = (
  (1.0 + 2.0 + 1.5) / 3,
  (2.0 + 1.5 + 2.5) / 3)
= (1.5, 2.0)
```

This centroid is not "the user". It is a compact representation of what the user is currently exploring, in this market, during this session.

Now, this centroid can be used to compare against other possible candidate listings. Airbnb can compute the cosine similarity between `Click centroid = (1.5, 2.0)` and `L4 = (1.6, 2.1)` and `L5 = (5.4, 3.2)` and select the one that looks the most similar to the user centroid.

Short-term intent and long-term preference

The same idea is reused across different time horizons, different actions and different granularities. Table 6 describes a different question that Airbnb is trying to answer:

Table 6: Embedding Features for Search Ranking

Feature Name	Description
EmbClickSim	similarity to clicked listings in H_c
EmbSkipSim	similarity to skipped listings H_s
EmbLongClickSim	similarity to long clicked listings H_{lc}
EmbWishlistSim	similarity to wishlisted listings H_w
EmbInqSim	similarity to contacted listings H_i
EmbBookSim	similarity to booked listing H_b
EmbLastLongClickSim	similarity to last long clicked listing
UserTypeListingTypeSim	user type and listing type similarity

(1) H_c: **clicked listing_ids** - listings that user clicked on in last 2 weeks.
(2) H_{lc}: **long-clicked listing_ids** - listing that user clicked and stayed on the listing page for longer than 60 sec.
(3) H_s: **skipped listing_ids** - listings that user skipped in favor of a click on a lower positioned listing
(4) H_w: **wishlisted listing_ids** - listings that user added to a wishlist in last 2 weeks.
(5) H_i: **inquired listing_ids** - listings that user contacted in last 2 weeks but did not book.
(6) H_b: **booked listing_ids** - listings that user booked in last 2 weeks.

How are `H*` calculated?

Airbnb, for each user_id would collect and maintain sets of *listing IDs* (not embeddings) that match specific logics (these would be `Hc`, `Hlc`, `Hs`, `Hw`, `Hi`, `Hb` from above). Airbnb also calculated these listing sets by market. For example, if user had clicked on listings from New York and Los Angeles, their set `Hc` would be further split into `Hc(NY)` and `Hc(LA)`.

How are `Emb[action]` (like `EmbClickSim`) calculated?

Let's now walk through the computation of one concrete feature: `EmbClickSim` *(the same process below applies to all other* `Emb[action]`*)*

This feature answers a very specific question: *Does this candidate listing look like what the user has been clicking on recently, in this market?*

The computation happens in 4 steps.

1. **Start with listing embeddings.** We know the listing IDs in `Hc`. We also know the embeddings of each of these listing IDs.
2. **Compute market-level centroids for `Hc`.** Suppose the user's clicked listings in NY and LA. Then Airbnb would compute 2 centroids based on the embeddings of the relevant listings:
   a. `Hc(NY) = {L1, L2, L3, L4, L5} → μ_Hc(NY)`
   b. `Hc(LA) = {L6, L7, L8) → μ_Hc(LA)`
   c. Each centroid summarises *what the user has been clicking on in that market*.
3. **Compute cosine similarities to a candidate listing.** Now, take a possible candidate listing `l(i)` with, of course, its own embedding `v_l(i)`. We can compute now the cosine similarity between `v_l(i)` and the market centroids.
   a. cosine similarity(`v_l(i)`, `μ_Hc(NY)`)
   b. cosine similarity(`v_l(i)`, `μ_Hc(LA)`)
4. **Take the maximum similarity (Equation 10).**

$$\text{EmbClickSim}(l_i) = \max_{m \in \mathcal{M}} \cos\left(v_{l_i}, \mu_{H_c}^{(m)}\right)$$

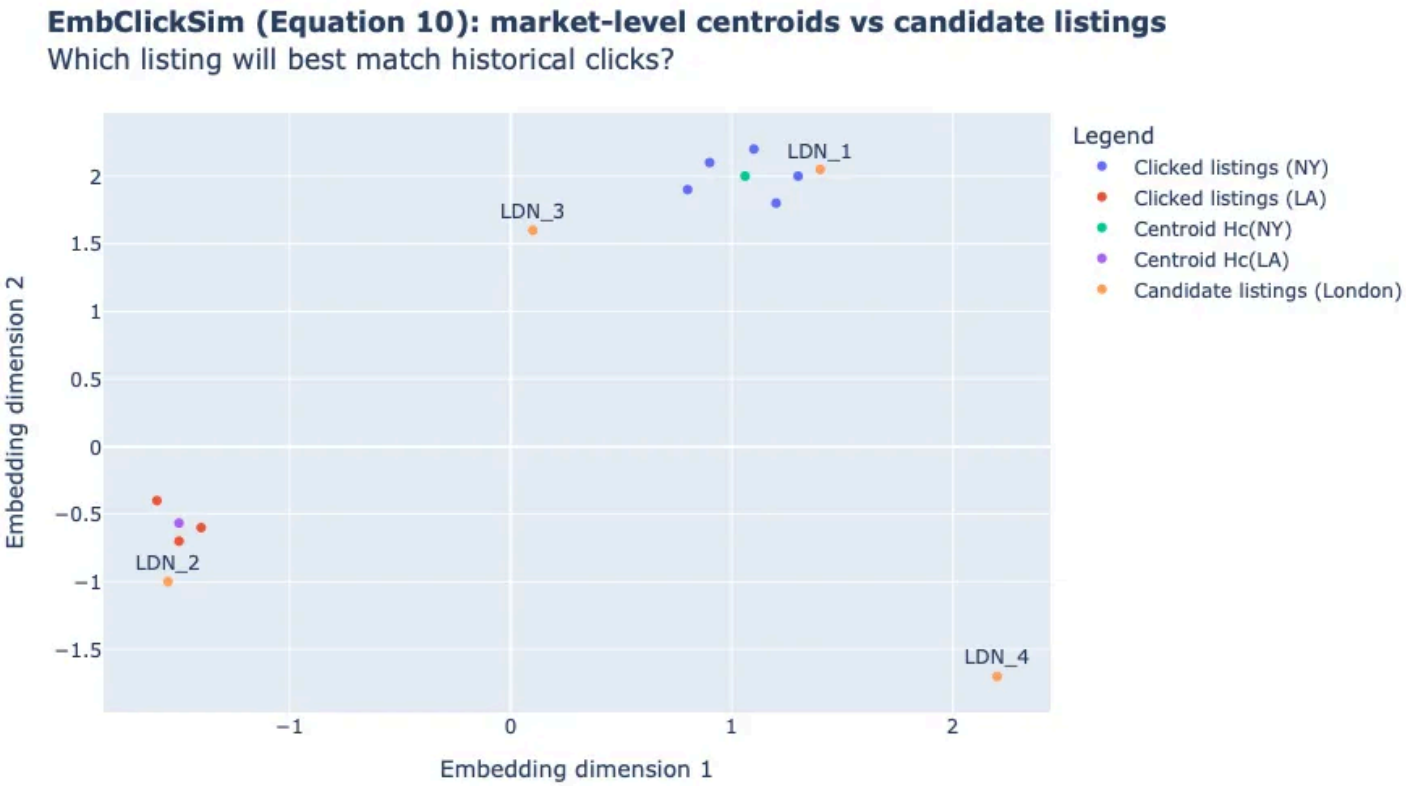
where:

- `M` is the set of markets in which the user has recent clicks
- `μ_Hc(m)` is the centroid of clicked listings in market `m`

The output of this cosine similarity between 2 embedding vectors would be 1 scalar number, which answers **one interpretable question** about user behaviour.

We can see this visually with 2 mock up plots:

1. The plot below contains the clicked listings and centroid for NY and LA. It also shows in orange possible candidate listings to show the traveller when he/she looks for London.



2. The table below then shows the numerical calculations of computing the market level centroids, the market level cosine similarities and the output of equation (10) for all 4 London candidate listings.

EmbClickSim example (Equation 10): cosine similarity to market-level click centroids

Candidate listing	$\cos(\ell_i, \mu_h c(NY))$	$\cos(\ell_i, \mu_h c(LA))$	$\text{EmbClickSim} = \max_m \cos(\ell_i, \mu_h c(m))$	Best matching market	Rank
LDN_1	0.994	-0.819	**0.994**	NY	1
LDN_2	-0.873	0.978	**0.978**	LA	2
LDN_3	0.911	-0.411	**0.911**	NY	3
LDN_4	-0.170	-0.524	**-0.170**	NY	4

EmbClickSim is computed as the maximum cosine similarity across market-level click centroids (Equation 10).

Where this leaves us

By now, the role of embeddings should be very clear:

- Skip-gram learns *geometry*
- Centroids summarise *behaviour*
- Cosine similarity turns geometry into *signals*

Only in the next section does Airbnb ask:

> *Given all these signals, how should we rank listings?*

And that is where the ranking model finally enters.

How embeddings moved from representations to ranking features.

Up to this point, we have focused on how Airbnb learns and aggregates embeddings to produce meaningful behavioural signals. The final step is to decide **how much each of these signals should matter** when ranking listings. This is where the ranking model enters.

What the model is trying to predict

Airbnb formulates search ranking as a supervised learning problem.

For each search, the training data consists of pairs `(xi,yi)`, where `xi` is the feature vector for the i-th listing shown to the user, and `yi` is a **graded outcome** observed after the search.

Rather than predicting bookings directly, Airbnb assigns utility values based on what eventually happened:

- `yi=1` if the listing was booked
- `yi=0.25` if the host was contacted but no booking occurred
- `yi=0.01` if the listing was clicked

- `yi=0` if the listing was viewed but not clicked
- `yi=-0.4` if the host rejected the guest

What goes into the feature vector `xi`

The feature vector combines several families of features:

- **Listing features**: price, capacity, room type, rejection rate, etc.
- **Query features**: dates, number of guests, length of stay, lead time.
- **User features**: historical averages such as booked price or guest rating.
- **Cross-features**: interactions between user, listing and query (e.g. capacity fit, price difference, rejection probability).
    - The embedding features we discussed earlier appear here as **cross-features**.
    - Remember, `EmbClickSim` is no longer a vector of embeddings… it was a scalar.

The ranking model

Airbnb uses a **Gradient Boosted Decision Tree** (**GBDT**) model trained with a LambdaRank objective. Offline evaluation is done using NDCG on held-out search sessions, and models are retrained regularly using the most recent 30 days of data.

What actually mattered in practice

To understand whether embedding features were doing useful work, Airbnb examined both **feature coverage** and **feature importance**.

Two patterns stand out.

First, short-term embedding features have the highest coverage, as expected. Clicks and skips are abundant, while bookings are sparse.
Second, embedding features rank surprisingly high among all features:

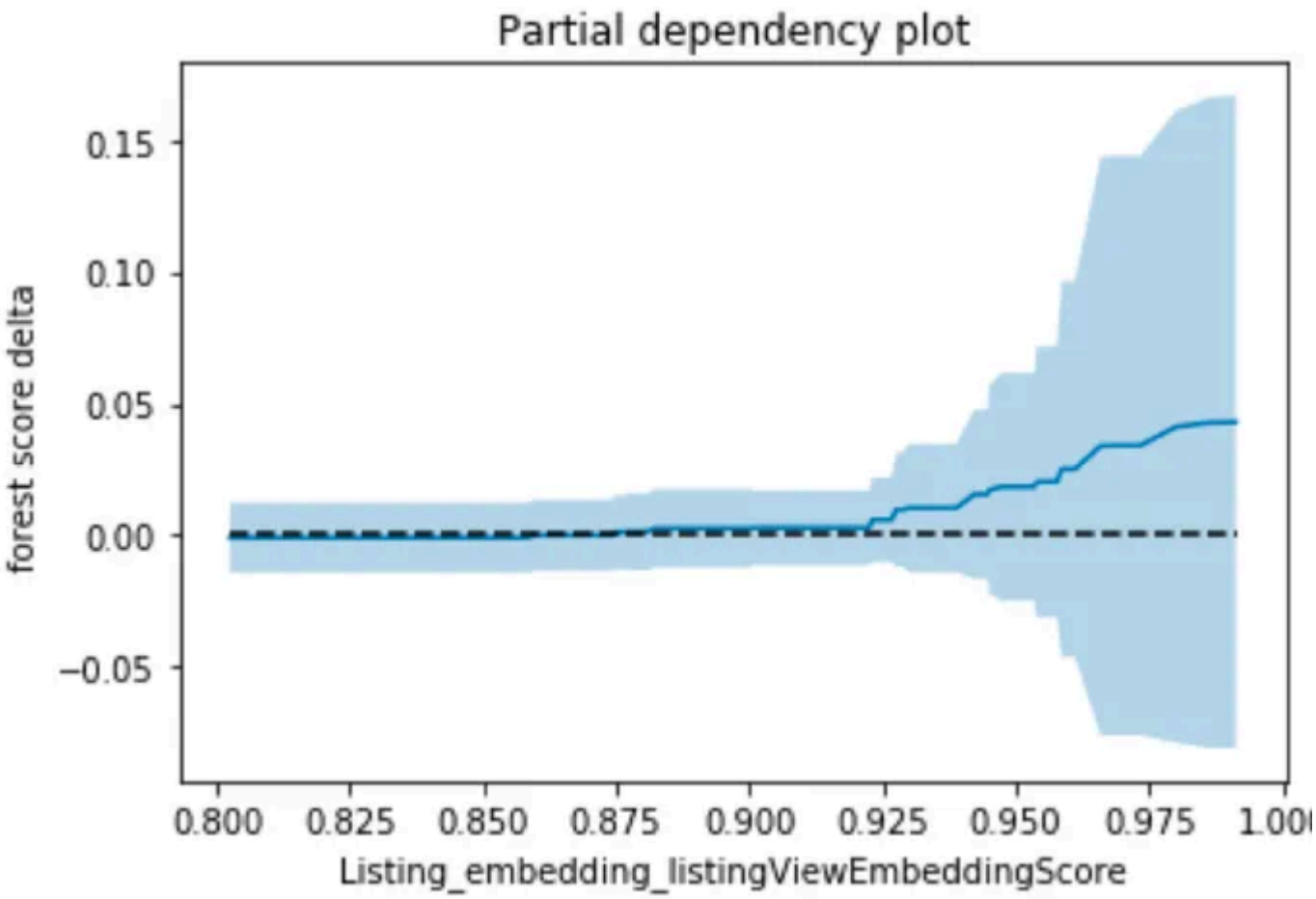
- `EmbClickSim` ranks **5th overall**
- `EmbSkipSim` ranks **8th overall**
- Five embedding features appear in the top 20 out of ~100 features

Table 7: Embedding Features Coverage and Importances

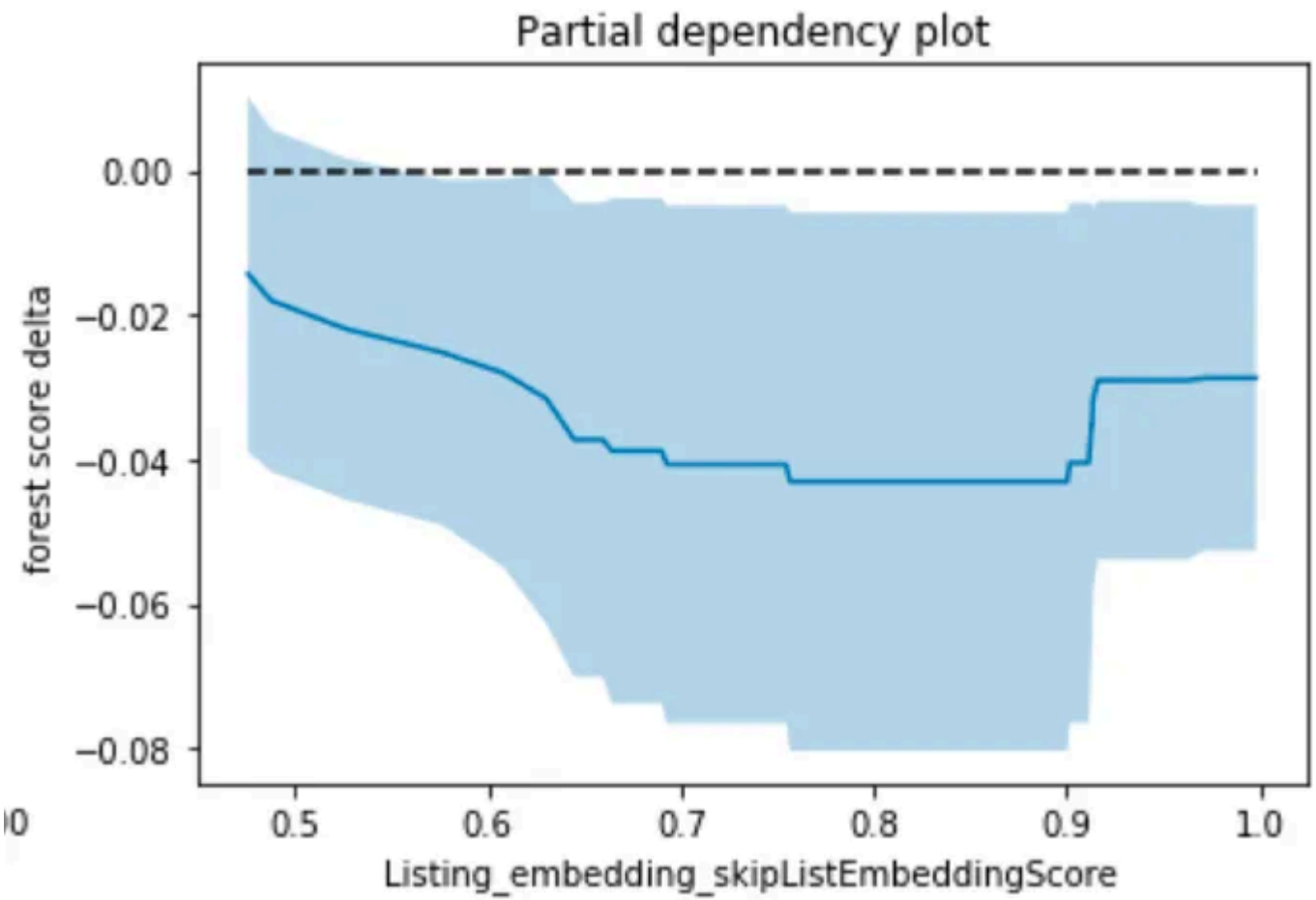
Feature Name	Coverage	Feature Importance
EmbClickSim	76.16%	**5/104**
EmbSkipSim	78.64%	**8/104**
EmbLongClickSim	51.05%	20/104
EmbWishlistSim	36.50%	47/104
EmbInqSim	20.61%	12/104
EmbBookSim	8.06%	46/104
EmbLastLongClickSim	48.28%	11/104
UserTypeListingTypeSim	86.11%	22/104

Partial dependence plots confirm that the model uses these features as intended:

- higher `EmbClickSim` increases ranking score



- higher `EmbSkipSim` decreases ranking score



In other words, the ranking model learns to trust embedding-based signals in exactly the way the geometry suggests.

Summary

Across these 2 posts, we followed a single idea all the way from raw user behaviour to production ranking.

In Part 1, we focused on listings.

- Click sessions were treated like sentences.
- Skip-gram turned co-occurrence into geometry.
- Listing embeddings were useful on their own through simple cosine similarity.
- *Similar Listings* worked because the representation was good, not because the model was complex.

In Part 2, we turned to users and personalisation.

- Sparse bookings and delayed feedback made user modelling harder.
- Airbnb introduced user types and listing types to make learning feasible.
- Booking sequences and host rejections shaped a shared embedding space.
- The geometry captured not just what worked, but also what failed.

What is interesting about Airbnb's approach is not the sophistication of the ranking model (it was a simple GBT model), but the discipline of the system design.

Embeddings are trained separately, aggregated simply, and injected into the ranking model as first-class features. The ranking model then learns how to combine short-term intent, long-term preference, and classical features — without any end-to-end black-box optimisation.

The result is a system that is: expressive, scalable and, crucially, explainable

With this, the full loop is closed: from raw user behaviour, to embeddings, to behavioural signals, to production ranking.

📢 Now I want to hear from you!

If you made it this far, thank you for reading **both parts** of this deep dive.
We covered a lot of ground — from learning representations with skip-gram, to shaping embedding spaces with sparse signals, and finally to turning geometry into production-ready ranking signals.

I would be genuinely curious to hear how this compares to your own experience.

- Have you worked with **embeddings or similarity models in production**, rather than only in offline experiments?

- How do you decide which **user interactions should matter most** in your system — clicks, bookings, rejections, or something else?
- When evaluating representation learning approaches, do you rely on **offline metrics and visualisations**, or do you only trust what survives an **online experiment**?
- And if you have shipped embeddings into a live system, how did you **validate that they were actually helping** — and where did things break in ways you did not expect?

Drop your thoughts or questions in the comments.

I am interested in how others approach representation learning and ranking at scale… especially in messy, real-world systems.

Thanks again for reading!

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

8 Likes · 2 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Adeline Latruwe The Cognitive Stack 13 Mar

Liked by Jose Parreño Garcia

Loved the deep dive! That k you for sharing.

once it clicks you're in a geometric space, I love how versatile embeddings can be. So much more we can do, like tracking behavior, inferring ranking...way beyond simple fact storage.

LIKED (1) REPLY SHARE

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture